

Reviewer Pack — Minimal Order of Formal Contexts and DPLL Proof Width Inequa...

Entry 9da834077d87 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 16:10:14 UTC

Minimal Order of Formal Contexts and DPLL Proof Width Inequality

Entry ID: 9da834077d87 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 16:10:14 UTC

1. Conjecture

Field A (mathematical branch): Lattice Theory (Formal Contexts) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For every Conjunctive Normal Form (CNF) instance φ with n variables, the minimal order of a formal context associated with φ is linearly correlated with its DPLL proof width $w(\varphi)$, such that the minimal order of the formal context $= \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Formal contexts capture the relationships between elements in a set through a binary relation. They can be used to represent the structure of CNF instances, and the minimal order of a formal context could potentially reflect the complexity of finding a satisfying assignment, as indicated by the DPLL proof width.

Taxonomy category: TSEITIN (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1088a382090b1592

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the computed minimal order of the formal context for each CNF instance is within a linear relationship with its DPLL proof width, as indicated by a Pearson correlation coefficient greater than or equal to 0.8 across 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math
from itertools import combinations, product

def generate_cnf(n):
    clauses = []
    for _ in range(2 * n):
        literals = [f'x{i+1}' if random.choice([True, False]) else f'-x{i+1}' for i in range(n)]
        clauses.append(' AND '.join(literals))
    return ' OR '.join(clauses)

def construct_formal_context(cnf):
    variables = set()
    relation = {}

    for clause in cnf.split(' OR '):
        literals = clause.split(' AND ')
        for literal in literals:
            variables.add(literal[2:] if literal.startswith('-') else literal)
            if literal not in relation:
                relation[literal] = set()
            for other_literal in literals:
                if literal != other_literal:
                    relation[literal].add(other_literal)

    return variables, relation

def is_transitively_closed(relation):
    closure = set(relation.keys())
    while True:
        new_closure = closure.copy()
        for x, y_set in relation.items():
            for y in y_set:
                if y not in closure and any(z in closure for z in relation.get(y, [])):
                    new_closure.add(y)
        if new_closure == closure:
            break
```

```

        break
    closure = new_closure

    return closure == set(relation.keys())

def minimal_order(variables, relation):
    max_subcontext_size = 0
    for subset in combinations(variables, len(variables) // 2 + 1):
        subcontext_relation = {x: relation[x] & set(subset) for x in subset if x in relation}
        if is_transitively_closed(subcontext_relation):
            max_subcontext_size = max(max_subcontext_size, len(subset))
    return max_subcontext_size

def dpll(cnf):
    literals = cnf.split(' AND ')
    assignment = {}

    def solve(literals, assignment):
        if not literals:
            return True
        literal = next((l for l in literals if l[0] != '-'), None)
        if literal is None:
            return False

        new_assignment = assignment.copy()
        new_assignment[literal] = True
        if solve([l for l in literals if l != literal], new_assignment):
            return True

        new_assignment[literal] = False
        if solve([l for l in literals if l != f'-{literal}'], new_assignment):
            return True

        return False

    return solve(literals, assignment)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    metrics = []
    instances_tested = 0
    n_max = 0

    for n in n_values:
        for _ in range(5):
            cnf = generate_cnf(n)
            variables, relation = construct_formal_context(cnf)
            order = minimal_order(variables, relation)
            width = dpll(cnf)

            metrics.append({
                'n': n,
                'order': order,
                'width': width
            })

```

```

    })
    instances_tested += 1
    n_max = max(n_max, n)

if not metrics:
    return {
        "metric_name": "Minimal Order of Formal Contexts and DPLL Proof Width Inequality",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

order_values = [m['order'] for m in metrics]
width_values = [m['width'] for m in metrics]

mean_order = sum(order_values) / len(order_values)
mean_width = sum(width_values) / len(width_values)

correlation_coefficient = (sum((x - mean_order) * (y - mean_width) for x, y in zip(order_values, width_values)) /
                           math.sqrt(sum((x - mean_order) ** 2 for x in order_values) *
                                       sum((y - mean_width) ** 2 for y in width_values)))

return {
    "metric_name": "Minimal Order of Formal Contexts and DPLL Proof Width Inequality",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": abs(correlation_coefficient) >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r['metric_value'] for r in results if r['metric_value'] is not None) / len(results)
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample='correlation_coefficient<0.8' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0650c4b.py", line 75, in solve
    if solve([l for l in literals if l != literal], new_assignment):
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0650c4b.py", line 75, in solve
    if solve([l for l in literals if l != literal], new_assignment):
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0650c4b.py", line 75, in solve
    if solve([l for l in literals if l != literal], new_assignment):
    ~~~~~
[Previous line repeated 2 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0650c4b.py", line 79, in solve
    if solve([l for l in literals if l != f'-{literal}'], new_assignment):
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0650c4b.py", line 79, in solve
    if solve([l for l in literals if l != f'-{literal}'], new_assignment):
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0650c4b.py", line 79, in solve
    if solve([l for l in literals if l != f'-{literal}'], new_assignment):
    ~~~~~
[Previous line repeated 987 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0650c4b.py", line 75, in solve
    if solve([l for l in literals if l != literal], new_assignment):
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0650c4b.py", line 69, in solve
    literal = next((l for l in literals if l[0] != '-'), None)
    ~~~~~
RecursionError: maximum recursion depth exceeded

```

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to compute the Pearson correlation coefficient. | next: Re-run the test with a different implementation or fix the recursion error in the code to verify the conjecture.

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14738
2	preregistration	ollama_remote	glm4:latest	0	0	9775
3	novelty	ollama_remote	glm4:latest	0	0	8625
4	novelty	ollama_remote	glm4:latest	0	0	8425

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29147
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10463
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12200
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15594
9	judge	ollama_remote	glm4:latest	0	0	12388

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 121356 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9da834077d87.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9da834077d87.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9da834077d87.tar.gz` (if generated)